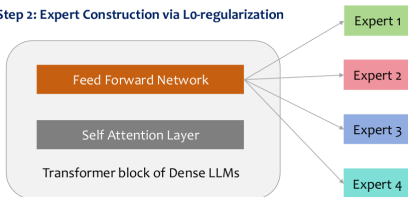


Accelerating Dense LLMs via L0-regularized Mixture-of-Experts

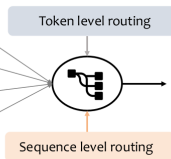
Step 1: Cluster Confusion Matrix based Sampling



Step 2: Expert Construction via L0-regularization



Step 3: Dynamic Batching for MoE Training



Zhenyu Zhang, Jiudong Yang, Zhaowen Tao, Meng Chen

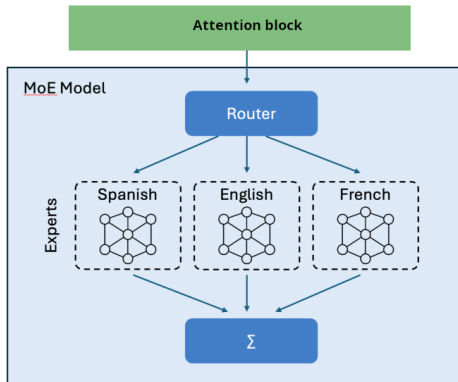
Context and Motivation

- LLMs showcase strong performance but are *costly at inference*.
- Exists acceleration methods include:
 - Quantization
 - Model pruning
 - Knowledge distillation

Context and Motivation

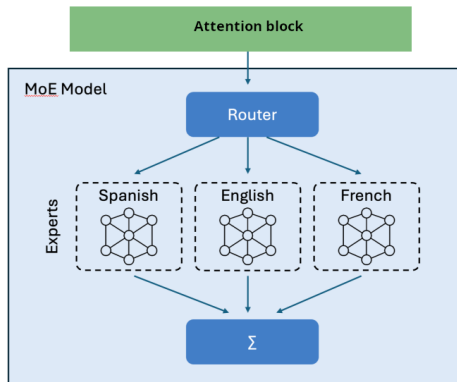
- LLMs showcase strong performance but are *costly at inference*.
- Exists acceleration methods include:
 - Quantization
 - Model pruning $\Rightarrow$ Performance degradation
 - Knowledge distillation

Mixture of Experts



- **Current MoE** training focuses on training from scratch or upcycling dense LLMs
⇒ requires vast computational resources and high quality corpora.

Mixture of Experts



- **L0-regularized MoE** (proposed method)
 - requires small-scale training corpus.
 - Does not require retraining the FFN weights.

L0-regularized MoE

Claims:

- Achieves up to a $2.5\times$ inference speed-up
- No significant performance loss across 4 benchmarks.

L0-regularized MoE

Claims:

- Achieves up to a $2.5\times$ inference speed-up
- No significant performance loss across 4 benchmarks.

Key components:

- 1 Sampling method to curate the training dataset into K semantic domains (on which experts will be extracted).

L0-regularized MoE

Claims:

- Achieves up to a $2.5\times$ inference speed-up
- No significant performance loss across 4 benchmarks.

Key components:

- 1 Sampling method to curate the training dataset into K semantic domains (on which experts will be extracted).
- 2 Extracting K experts by learning masks on FFN intermediary dimension.

L0-regularized MoE

Claims:

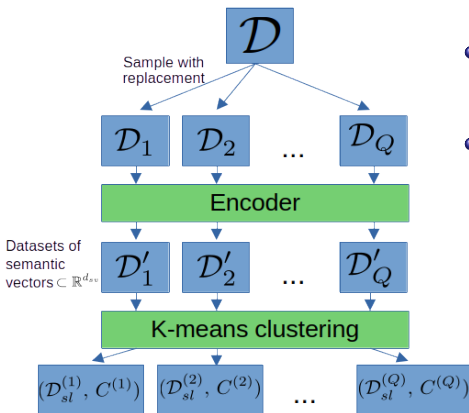
- Achieves up to a $2.5\times$ inference speed-up
- No significant performance loss across 4 benchmarks.

Key components:

- 1 Sampling method to curate the training dataset into K semantic domains (on which experts will be extracted).
- 2 Extracting K experts by learning masks on FFN intermediary dimension.
- 3 Learning a router to select experts.

Domain aware dataset curation

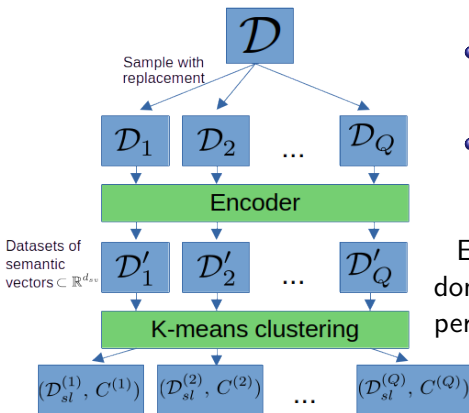
let $\mathcal{D}$ be the training dataset.



- $\mathcal{D}_{sl}^{(l)} = \{\mathcal{D}_{s_1}^{(l)}, \dots, \mathcal{D}_{s_K}^{(l)}\}$ the datasets obtained by clustering.
- $C^{(l)} = \{C_1^{(l)}, \dots, C_K^{(l)}\}$ the centroids.

Domain aware dataset curation

let $\mathcal{D}$ be the training dataset.

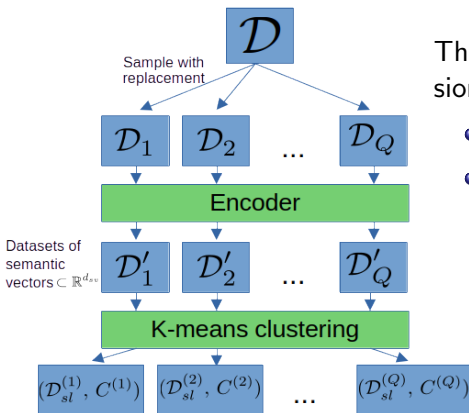


- $\mathcal{D}_{sl}^{(l)} = \{\mathcal{D}_{s_1}^{(l)} \dots, \mathcal{D}_{s_K}^{(l)}\}$ the datasets obtained by clustering.
- $C^{(l)} = \{C_1^{(l)}, \dots, C_K^{(l)}\}$ the centroids.

Each $\mathcal{D}_{sl}^{(l)}$ is reordered to align semantic domains before being passed to the experts.

Domain aware dataset curation: CCM

let $\mathcal{D}$ be the training dataset.

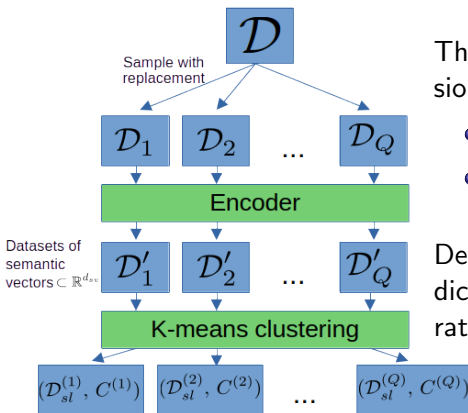


They use an algorithm called the Confusion Clustering Matrix (CCM) based on:

- an intra-clustering metric
- an inter-clustering metric

Domain aware dataset curation: CCM

let $\mathcal{D}$ be the training dataset.

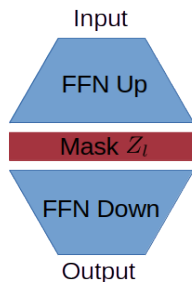


They use an algorithm called the Confusion Clustering Matrix (CCM) based on:

- an intra-clustering metric
- an inter-clustering metric

Derive for each $\mathcal{D}_{sl}^{(l)}$ a measure $d_{ds}^{(l)}$ indicating how well sub-datasets are separated by semantic domains.

Experts extraction



- We want to extract K experts on the K semantic domains
- For that for each semantic domain l , we want to learn a mask $Z_l \in \{0, 1\}^{d^{int}}$

The intermediate layer weights W are replaced by

$$W \odot Z_l$$

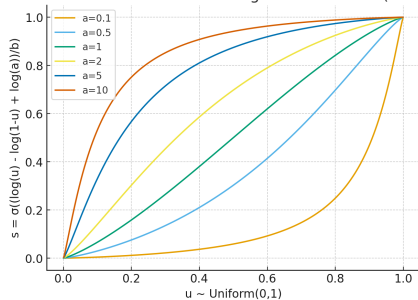
Experts extraction

- Problem: Z_I has no continuous values, so non-differentiable $\Rightarrow$ not learnable

Experts extraction

- Problem: Z_I has no continuous values, so non-differentiable $\Rightarrow$ not learnable
- use a stochastic, differentiable relaxation of Z_I :

Effect of a on Hard Concrete sigmoid transform ($b=0.83$)



$$u \sim \mathcal{U}(0, 1),$$

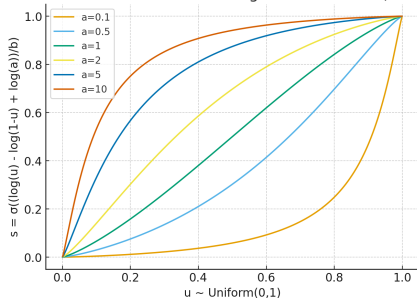
$$s = \text{Sigmoid}\left(\frac{\log u - \log(1 - u) + \log a}{b}\right),$$

$$\bar{s} = s(\zeta - \gamma) + \gamma,$$

$$z = \min(1, \max(0, \bar{s})).$$

Experts extraction

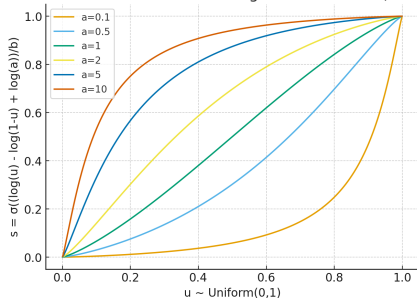
Effect of a on Hard Concrete sigmoid transform ($b=0.83$)



- Learn parameter a so that Z_l converges to a binary mask.
- we also want to control the proportion of retained weights:
$$\hat{r} = \frac{\sum Z_l}{d_{int}}$$

Experts extraction

Effect of a on Hard Concrete sigmoid transform ($b=0.83$)



- Learn parameter a so that Z_l converges to a binary mask.
- we also want to control the proportion of retained weights:
$$\hat{r} = \frac{\sum Z_l}{d_{int}}$$
- Constrained optimization via a Lagrangian penalty:
$$\mathcal{L}_{l_0} = \lambda_1(\hat{r} - r) + \lambda_2(\hat{r} - r)^2$$

$$\mathcal{L}_{exp} = \mathcal{L}_{LLM} + \mathcal{L}_{l_0} \text{ on } \mathcal{D}_{sl}^{(0)}$$

Router learning

The router is a matrix W_r that dynamically selects the expert processing each token.

We want the load on each expert to be balanced:

Router learning

The router is a matrix W_r that dynamically selects the expert processing each token.

We want the load on each expert to be balanced:

- $\mathcal{L}_{\text{balance}} = \sum_{i=1}^K \left(\frac{c_i}{B} - \frac{1}{K} \right)^2$
Each expert receives roughly the same amount of tokens (Macro balancing).

Router learning

The router is a matrix W_r that dynamically selects the expert processing each token.

We want the load on each expert to be balanced:

- $\mathcal{L}_{\text{balance}} = \sum_{i=1}^K \left(\frac{c_i}{B} - \frac{1}{K} \right)^2$
Each expert receives roughly the same amount of tokens (Macro balancing).
- $\mathcal{L}_z = \frac{1}{B} \sum_{i=1}^B \left(\log \left(\sum_{j=1}^K e^{x_i^{(j)}} \right) \right)$
Each token is more uniformly distributed over multiple experts (Micro balancing).

Router learning

$$\mathcal{L}_{\text{rout}} = \mathcal{L}_{LLM} + \alpha \mathcal{L}_{\text{balance}} + \beta \mathcal{L}_z$$

This is trained on $(\mathcal{D}_{sl}^{(l)})_{1 \leq l \leq Q}$ in decreasing order of $d_{ds}^{(l)}$ (measures separability wrt semantic domains).

Router learning

$$\mathcal{L}_{\text{rout}} = \mathcal{L}_{\text{LLM}} + \alpha \mathcal{L}_{\text{balance}} + \beta \mathcal{L}_z$$

This is trained on $(\mathcal{D}_{sl}^{(l)})_{1 \leq l \leq Q}$ in decreasing order of $d_{ds}^{(l)}$ (measures separability wrt semantic domains).

- In the early iterations, the Mixture-of-Experts rapidly learns to route tokens toward the appropriate experts.

Router learning

$$\mathcal{L}_{\text{rout}} = \mathcal{L}_{\text{LLM}} + \alpha \mathcal{L}_{\text{balance}} + \beta \mathcal{L}_z$$

This is trained on $(\mathcal{D}_{sl}^{(l)})_{1 \leq l \leq Q}$ in decreasing order of $d_{ds}^{(l)}$ (measures separability wrt semantic domains).

- In the early iterations, the Mixture-of-Experts rapidly learns to route tokens toward the appropriate experts.
- As training progresses, the router is encouraged to distribute tokens across multiple experts.

Experimental results

Model	MMLU	GSM8K	HumanEval	BBH	Average	Speedup
Llama-3-8B	66.6	56.0	33.5	57.7	53.5	
Llama-3-8B w/ L0-MoE	66.3	55.9	33.7	57.2	53.3	2.0x
Mistral-7B	64.1	52.2	29.3	56.1	50.4	
Mistral-7B w/ L0-MoE	64.8	53.6	31.1	55.9	51.4	2.1x
Qwen2-7B	70.3	79.9	51.2	62.6	66.0	
Qwen2-7B w/ L0-MoE	70.4	80.5	52.0	61.5	66.1	2.5x

It's over

Thank you for your attention.

